

République Tunisienne

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Institut Supérieur des Études Technologiques de Sfax

Année Universitaire : 2024/2025

Rapport de Projet de Fin d'Études

Master Professionnel DSIR

Développement des Systèmes Informatiques et Réseaux

Option : GLNT

Application de Gestion des Académies Sportifs

Sport Academy Pro

Backend + Frontend + Chatbot ML + Service IA Scouting

Réalisé par :	Mortadha Boufaied
Encadrant académique :	[Nom à compléter]
Encadrant entreprise :	Farouk Abdelkarim
Organisme :	Farouk Abdelkarim Consulting / cody
Année universitaire :	2024–2025

2 mai 2026

Table des matières

1	Resume executif	1
1.1	Objectifs du travail	1
1.2	Resultats techniques principaux	1
1.3	Valeur du rapport	2
2	Introduction generale	3
2.1	Contexte	3
2.2	Problematique	3
2.3	Demarche	3
2.4	Structure du rapport	4
3	Sources documentaires exploitees	5
3.1	Sources backend (Spring Boot)	5
3.2	Sources frontend (Flutter)	5
3.3	Sources chatbot (Django + ML)	6
3.4	Methodologie de consolidation	6
4	Architecture globale de la solution	7
4.1	Vue d'ensemble	7
4.2	Schema logique	7
4.3	Principe de separation des frontends	8
4.4	Flux principaux	8
5	Architecture detaillee et diagrammes	9
5.1	Architecture globale du systeme	9
5.2	Architecture microservices	10
5.3	Flux de donnees principaux	10
5.4	Architecture de securite	11
5.5	Architecture de base de donnees	12
5.6	Architecture de deploiement	12
5.7	Monitoring et observabilite	14

6	Etude du backend Spring Boot	15
6.1	Stack technique	15
6.2	Configuration serveur et donnees	15
6.3	Securite et autorisations	16
6.4	Organisation fonctionnelle REST	16
6.5	Chat temps reel backend	16
6.6	Build, tests et qualite	17
6.7	Conteneurisation et CI/CD	17
6.8	Point d'attention DB	17
7	Etude du frontend Flutter	18
7.1	Vue generale	18
7.2	Deux points d'entree applicatifs	18
7.3	Navigation, roles et securisation UI	19
7.4	Theming et coherence visuelle	19
7.5	Internationalisation	19
7.6	Configuration reseau frontend	19
7.7	Ameliorations majeures du module chat	19
7.8	Ameliorations de la page contacts	20
7.9	Notes de compatibilite	20
8	Etude du chatbot Django + ML	21
8.1	Objectif du chatbot	21
8.2	Stack technique	21
8.3	Exposition API	21
8.4	Moteur de reponse	22
8.5	Configuration et tuning	22
8.6	Securite API	22
8.7	Integration avec Flutter	22
9	Service IA et Scouting	23
9.1	Vue d'ensemble	23
9.2	Stack technique	23
9.3	Architecture du service	23
9.4	Fonctionnalites ML	24
9.5	Fonctionnalites Scouting	25
9.6	Integration avec football-academy	26
9.7	Tests et qualite	26
9.8	Deploiement	27

10 Fonctionnalites Multi-sport	28
10.1 Concept et architecture	28
10.2 Principes de conception	28
10.3 Implementation technique	29
10.4 Fonctionnalites multi-sport	31
10.5 Exemples d'implementation	31
10.6 Avantages de l'approche multi-sport	32
10.7 Integration avec les services IA	32
11 Integration et deploiement	33
11.1 Integration inter-projets	33
11.2 Variables de configuration	33
11.3 Strategie de deploiement recommandee	34
11.4 Procedure de lancement local	34
11.5 Points de vigilance en production	35
12 Tests et assurance qualite	36
12.1 Qualite backend	36
12.2 Qualite frontend	36
12.3 Qualite chatbot	36
12.4 Scenario de validation bout en bout	37
12.5 Indicateurs de suivi proposes	37
13 Limites actuelles et pistes d'amelioration	38
13.1 Limites identifiees	38
13.2 Ameliorations prioritaires	38
13.3 Evolutions fonctionnelles chat/chatbot	39
13.4 Perspective architecture	39
14 Conclusion generale	40
15 Annexes	41
15.1 Annexe A - Commandes utiles	41
15.2 Annexe B - Exemple de payload chatbot	41
15.3 Annexe C - Emplacements a completer dans le rapport	42

Liste des tableaux

5.1	Services et responsabilites	10
5.2	Roles et permissions	12
6.1	Synthese stack backend	15
9.1	Synthese stack service IA	23
10.1	Configuration Football	31
10.2	Configuration Basketball	32
11.1	Variables principales de deploiement	33
12.1	KPIs qualite recommandes	37

Chapitre 1

Resume executif

Ce document presente le rapport technique d'un ecosysteme logiciel compose de trois projets relies :

- un backend Java Spring Boot pour la gestion metier de l'academie ;
- un frontend Flutter pour mobile et web admin ;
- un chatbot Django base sur une recherche de reponses par similarite.

Le contenu est construit a partir des documents et fichiers techniques existants dans les repertoires `football-academy`, `moez_project` et `chatbot`.

1.1 Objectifs du travail

Les objectifs couverts par ce rapport sont les suivants :

- documenter l'architecture globale et les interactions entre services ;
- decrire les choix techniques backend, frontend et chatbot ;
- presenter les mecanismes de securite, de communication temps reel et de deploiement ;
- synthese des limitations connues et des pistes d'amelioration.

1.2 Resultats techniques principaux

- Backend Spring Boot 3.2.4, Java 17, securise par JWT, avec endpoints REST et Web-Socket STOMP.
- Frontend Flutter avec deux points d'entree : application mobile utilisateur et plateforme web admin.
- Chat interne optimise (pattern singleton, pre-subscriptions, mise a jour temps reel).
- Chatbot Django expose via `POST /api/chat`, avec pipeline exact match, fuzzy match et TF-IDF.
- Base de deploiement conteneurisee du backend via Docker et CI/CD GitLab.

1.3 Valeur du rapport

Ce projet LaTeX fournit une base operationnelle pour votre rapport PFE : vous pouvez le compiler immediatement sur Overleaf ou en local, puis remplacer progressivement les elements de personnalisation (noms, captures d'ecran, diagrammes, resultats de tests).

Chapitre 2

Introduction generale

2.1 Contexte

Le besoin metier vise une plateforme unifiee pour une academie de football, couvrant :

- la gestion administrative et operationnelle (joueurs, entraineurs, activites, paiements, notifications) ;
- l’experience utilisateur mobile (consultation, chat, profil, statistiques, recherche) ;
- une interface web admin dediee ;
- un assistant conversationnel pour les questions frequentes des utilisateurs.

Cette combinaison impose une architecture multi-projets avec interconnexion claire, separation des responsabilites et securisation des acces.

2.2 Problematique

Les principales questions techniques traitees dans ce rapport sont :

- comment organiser le backend pour supporter a la fois API REST, temps reel et controle d’accès par rôles ;
- comment structurer un frontend Flutter qui sert mobile et web admin sans duplication inutile ;
- comment integrer un chatbot local independant et interopere avec l’application mobile ;
- comment preparer un deploiement robuste et evolutif.

2.3 Demarche

La demarche retenue est basee sur l’analyse documentaire et du code existant :

- extraction des technologies, configurations et flux depuis les fichiers de configuration et README ;

- cartographie des routes, couches logicielles et dependances ;
- consolidation en chapitres exploitables pour un rapport academique.

2.4 Structure du rapport

Le rapport est organise en chapitres progressifs :

1. sources documentaires exploitees ;
2. architecture globale de la solution ;
3. etude du backend ;
4. etude du frontend ;
5. etude du chatbot ;
6. integration, deploiement, qualite et perspectives.

Chapitre 3

Sources documentaires exploitees

Ce rapport est base sur les documents techniques et fichiers de configuration suivants.

3.1 Sources backend (Spring Boot)

Fichier	Apport principal
football-academy/pom.xml	Stack backend, dependances, tests, couverture, qualite.
football-academy/src/main/resources/application.properties	Configuration de l'application, JWT, uploads, JPA.
football-academy/Dockerfile	Build multi-stage et execution du jar.
football-academy/docker-compose.yml	Variables d'environnement et exposition du service.
football-academy/.gitlab-ci.yml	Pipeline CI/CD (test, build, docker push).
football-academy/HELP.md	Notes de reference Maven/Spring Boot.
football-academy/src/README-ONE-DB.md	Verif de l'absence d'une importante sur l'usage d'une vraie base DB.
football-academy/uploads/README.txt	Strategie de stockage des images uploads.

3.2 Sources frontend (Flutter)

Fichier	Apport principal
moez_project/pubspec.yaml	SDK, dependances Flutter, bibliotheques reseau/chat/maps.
moez_project/lib/main.dart	Point d'entree mobile, routes principales.

Fichier	Apport principal
moez_project/lib/main_admin_web.d	Point d'entree web admin, role guard admin.
moez_project/lib/components/Constants.d	URLs backend REST/WS et URL chatbot.
moez_project/lib/l10n/app_strings.d	Localisation multilingue (en, fr, ar).
moez_project/CHAT_SYSTEM_IMPROVEMENTS.md	Evolution architecture chat temps reel.
moez_project/CONTACT_PAGE_IMPROVEMENTS.md	Engagement et UX pour contacts role-based.
moez_project/lib/README_LIB_UPDATES.md	Evolution resolution URL images/API.
moez_project/lib/PATCH_NOTES.md	Correctifs historises (chat, popup, search).
moez_project/lib/README_PLATFORM_LIMITS.txt	Limites plateforme (voice scrolling system-wide).

3.3 Sources chatbot (Django + ML)

Fichier	Apport principal
chatbot/chatbot/README.md	Objectif, endpoints, execution locale, tuning.
chatbot/chatbot/requirements.txt	Dependencies Django + stack data science.
chatbot/chatbot/chatbot/settings.py	Variables d'env, dataset QA, seuils similarity.
chatbot/chatbot/apps/chat/urls.py	Routage web et API chat.
chatbot/chatbot/apps/chat/views.py	Contrat API /api/chat et gestion cle API.
chatbot/chatbot/apps/chat/services/ml_indexer.py	Indexation TF-IDF + fuzzy + exact.

3.4 Methodologie de consolidation

L'information retenue privilegie :

- les fichiers de configuration executable (source de verite technique),
- les notes d'amelioration qui decrivent les decisions d'architecture,
- les points d'entree applicatifs pour decrire les flux utilisateur.

Chapitre 4

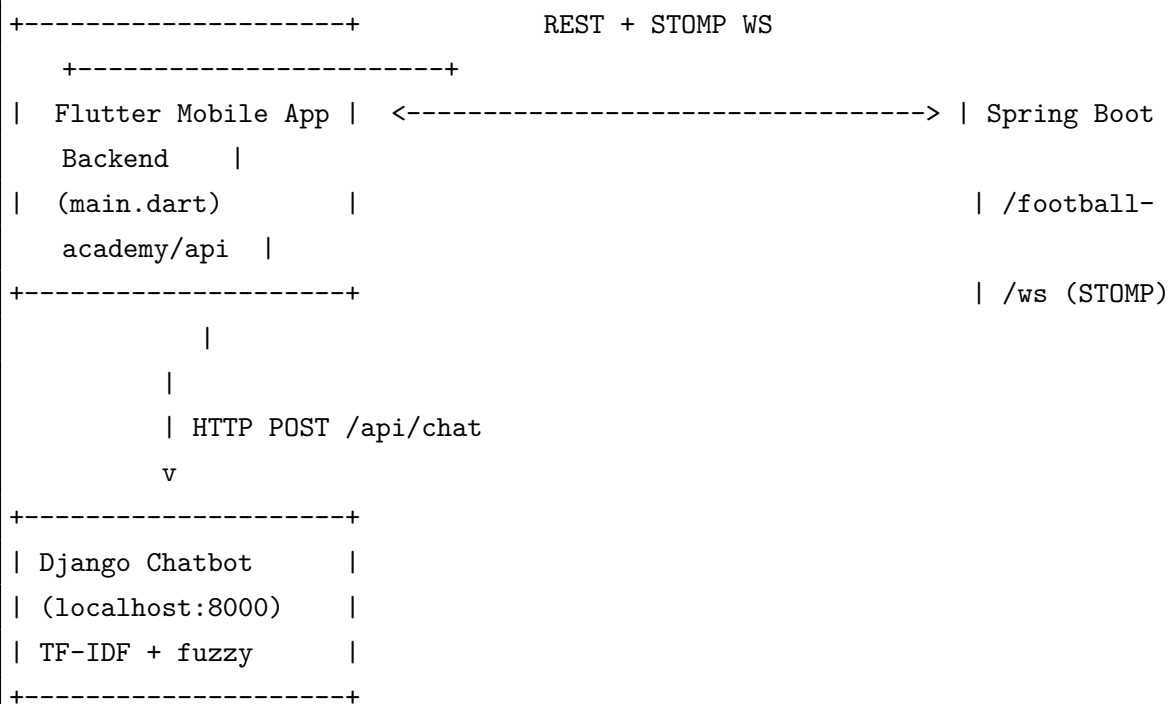
Architecture globale de la solution

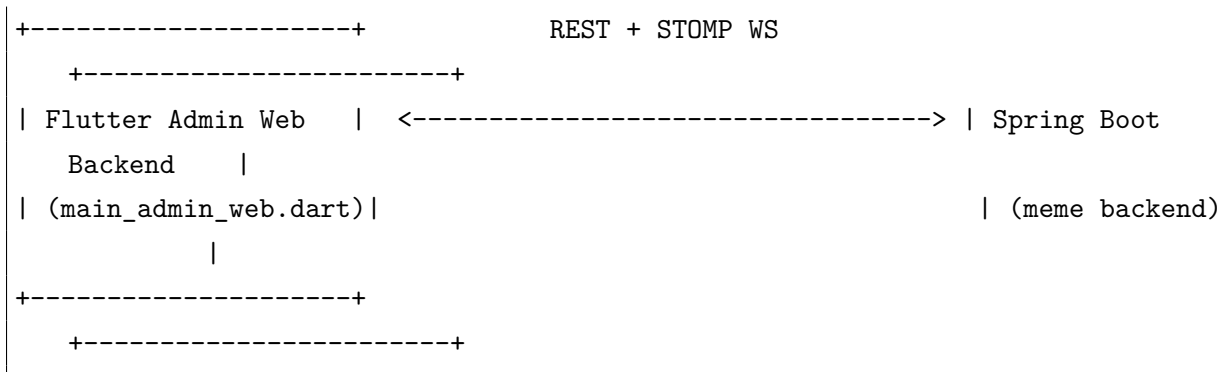
4.1 Vue d'ensemble

La solution est organisée autour de trois blocs indépendants mais connectés :

- **Backend** football-academy : API métier, sécurité JWT, temps réel WebSocket, stockage fichiers.
- **Frontend** moez_project : application Flutter mobile + portail web admin.
- **Chatbot** chatbot : service Django/ML exposé via endpoint HTTP local.

4.2 Schema logique





4.3 Principe de separation des frontends

Le frontend contient deux points d'entree distincts :

- `lib/main.dart` pour l'application mobile utilisateur ;
- `lib/main_admin_web.dart` pour la plateforme admin web.

Cette approche permet de deployer les deux interfaces separement sans imposer les memes contraintes de distribution.

4.4 Flux principaux

Flux d'authentification

1. l'utilisateur envoie ses credentials via l'app Flutter ;
2. le backend valide et emet un JWT ;
3. les routes sont protegees selon les roles (admin, trainer, etc.).

Flux de chat temps reel

1. le frontend charge conversations + contacts ;
2. le frontend s'abonne aux topics WebSocket ;
3. les nouveaux messages arrivent en push et mettent a jour UI.

Flux chatbot

1. l'utilisateur pose une question dans l'ecran chatbot Flutter ;
2. Flutter appelle `POST /api/chat` du service Django ;
3. Django retourne reponse + score + metadata de correspondance.

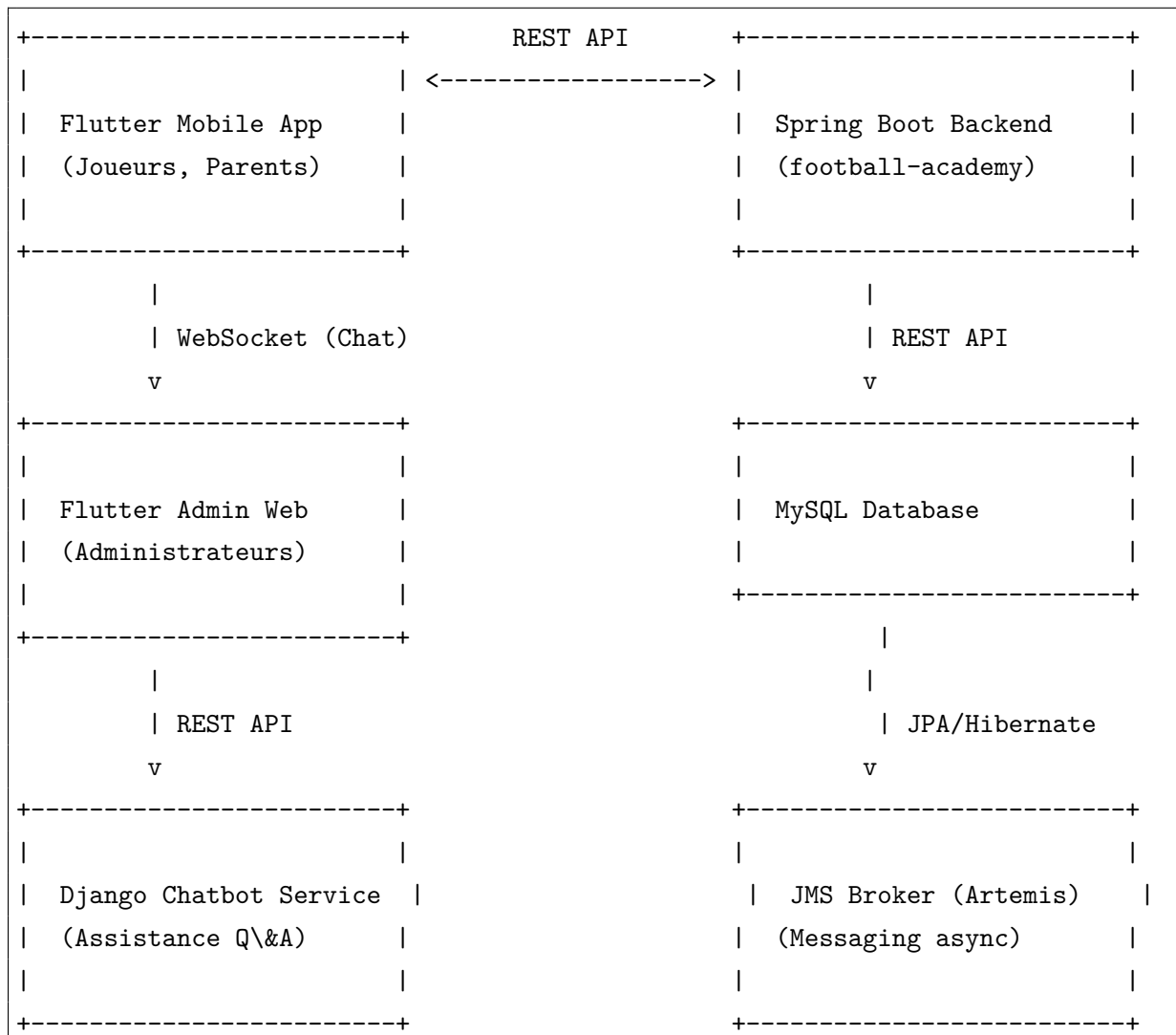
Chapitre 5

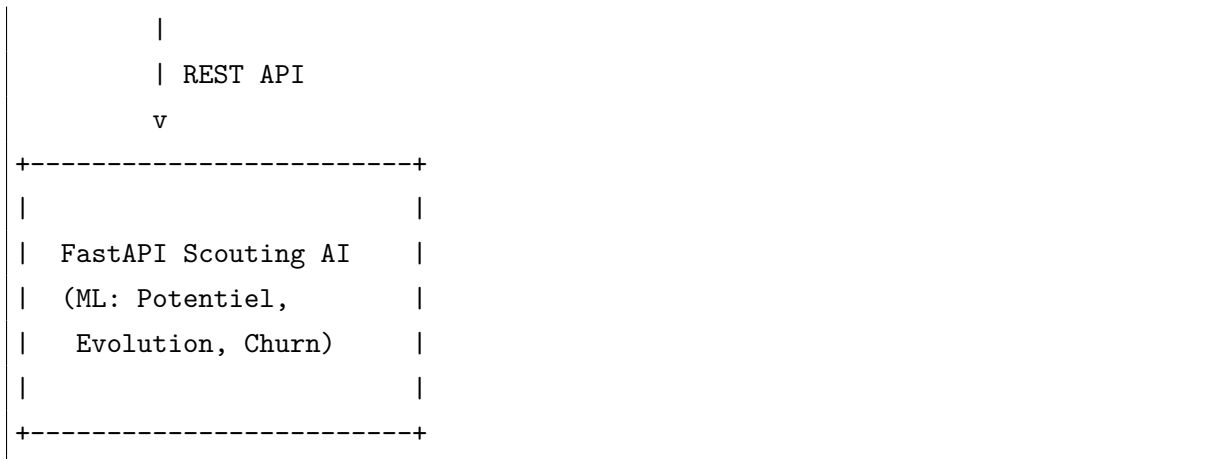
Architecture detaillee et diagrammes

5.1 Architecture globale du systeme

Vue d'ensemble des composants

L'écosysteme *Sport Academy Pro* est compose de plusieurs services interconnectes :





5.2 Architecture microservices

Separation des responsabilites

Chaque service a une responsabilite clairement definie :

Service	Responsabilite	Technologie
football-academy	API metier, authentication, gestion	Spring Boot
moez_project	Interface mobile et web admin	Flutter
chatbot	Assistance conversationnelle	Django/ML
scouting-ai	Analyse ML et scouting	FastAPI/ML
MySQL	Persistence des donnees	MySQL 8.0
Artemis	Messaging asynchrone	JMS

TABLE 5.1 – Services et responsabilites

5.3 Flux de donnees principaux

Flux d'authentification

1. L'utilisateur saisit ses identifiants dans l'app Flutter
2. Flutter envoie `POST /api/auth/login` au backend Spring Boot
3. Spring Boot valide les credentials et genere un JWT
4. Le JWT est retourne et stocke dans Flutter
5. Les requetes subsequentes incluent le JWT dans le header `Authorization: Bearer <token>`
6. Le `JwtAuthenticationFilter` valide le token a chaque requete

Flux de messagerie temps reel

1. L'utilisateur ouvre l'écran de chat dans Flutter
2. Flutter se connecte au WebSocket endpoint `/ws`
3. Flutter s'abonne aux topics STOMP (`/topic/messages/{conversationId}`)
4. Lorsqu'un message est envoyé :
 - Flutter envoie `POST /api/chat/messages`
 - Le backend sauvegarde le message en base
 - Le backend publie le message sur le topic JMS
 - Le WebSocket broker diffuse le message aux abonnées
5. Les clients abonnées reçoivent le message en temps reel

Flux de synchronisation IA

1. Le service scouting-ai appelle `POST /api/v1/data/sync/football-academy`
2. Il récupère les données joueurs depuis `/football-academy/api/players`
3. Il récupère les données paiements depuis `/football-academy/api/payments`
4. Les données sont stockées dans la base du service IA
5. Les modèles ML sont entraînés ou mis à jour
6. Les scores de potentiel, évolution et churn sont calculés

5.4 Architecture de sécurité

Couches de sécurité

- **Authentification** : JWT avec expiration et refresh
- **Autorisation** : RBAC (Role-Based Access Control)
- **Validation** : Validation des entrées et sanitization
- **Encryption** : Mots de passe hachés avec bcrypt
- **HTTPS** : Communication sécurisée (en production)

Roles et permissions

Role	Permissions
ADMIN	Acces total, gestion utilisateurs, configuration globale
TRAINER	Gestion equipes, planification, saisie statistiques
PARENT	Consultation enfants, paiements, planning
PLAYER	Consultation profil, planning, communication
SCOUTER	Recherche joueurs, comparaison, shortlists (lecture seule)

TABLE 5.2 – Roles et permissions

5.5 Architecture de base de donnees

Schema principal

La base de donnees MySQL est organisee autour des entites principales :

- **Users** : Utilisateurs du systeme
- **Players** : Profils des joueurs
- **Teams** : Equipes et divisions
- **Activities** : Entrainements, matchs, evenements
- **Payments** : Transactions et cotisations
- **Conversations** : Discussions et messages
- **Notifications** : Alertes et rappels

Relations cles

```
User (1) ----< (N) Player
Team (1) ----< (N) Player
Activity (1) ----< (N) Attendance
Player (1) ----< (N) Payment
User (1) ----< (N) Conversation
Conversation (1) ----< (N) Message
```

5.6 Architecture de deploiement

Docker Compose

L'ensemble des services est deploye via Docker Compose :

```
version: '3.8'

services:
  mysql:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: football_academy
    ports:
      - "3306:3306"

  football-academy:
    build: ./football-academy
    ports:
      - "8091:8091"
    depends_on:
      - mysql
      - artemis

  chatbot:
    build: ./chatbot
    ports:
      - "8000:8000"

  scouting-ai:
    build: ./scouting-ai-service
    ports:
      - "8010:8010"
    depends_on:
      - football-academy

  artemis:
    image: apache/activemq-artemis
    ports:
      - "61616:61616"
```

5.7 Monitoring et observabilite

Logs et traces

- Logs applicatifs structure (JSON)
- Traces des requetes API
- Monitoring des performances
- Alertes sur erreurs et anomalies

Health checks

Chaque service expose un endpoint de health check :

GET `/health` pour football-academy

- GET `/api/health` pour scouting-ai
- Verification de la connexion base de donnees
- Verification des dependances externes

Chapitre 6

Etude du backend Spring Boot

6.1 Stack technique

Le backend est un projet Spring Boot (parent `spring-boot-starter-parent` version 3.2.4) avec Java 17.

Composant	Technologie
Framework principal	Spring Boot 3.2.4
Langage	Java 17
Web/API	spring-boot-starter-web
Data access	spring-boot-starter-data-jpa
Securite	spring-boot-starter-security + JWT (jjwt)
Validation	spring-boot-starter-validation
Temps reel	spring-boot-starter-websocket + STOMP
Documentation API	springdoc-openapi-starter-webmvc-ui
Messaging interne	spring-jms + Artemis Jakarta
Base de donnees	MySQL (driver 8.0.33)

TABLE 6.1 – Synthese stack backend

6.2 Configuration serveur et donnees

Le backend expose par default :

- `server.port=${PORT:8091}`;
- `server.servlet.context-path=/football-academy`.

La datasource configuree cible MySQL avec JPA/Hibernate. Le stockage des uploads repose sur `file.storage.location=uploads` et une exposition statique des ressources.

6.3 Sécurité et autorisations

JWT et filtres

La classe `JwtAuthenticationFilter` supporte :

- header `Authorization: Bearer <token>`;
- cookie `accessToken`;
- fallback paramètre query (debug).

Règles de sécurité

La `SecurityConfig` applique une politique stateless et distingue :

- chemins publics (auth, assets, uploads, websocket) ;
- routes admin restreintes à `ROLE_ADMIN` ;
- routes `/api/**` authentifiées.

6.4 Organisation fonctionnelle REST

Les contrôleurs REST couvrent plusieurs domaines métier :

- authentification et profil ;
- gestion activités/matches/calendar ;
- joueurs, entraîneurs, parents, divisions, paiements ;
- notifications ;
- recherche globale ;
- chat et contacts.

6.5 Chat temps réel backend

Le chat combine :

- endpoints REST `/api/chat/...` pour conversations/messages ;
- WebSocket STOMP endpoint `/ws` ;
- broker simple avec préfixes `/topic` et `/queue` ;
- file JMS interne pour l'envoi asynchrone des messages.

Cette architecture favorise la fluidité de l'interface chat et limite les rafraichissements manuels.

6.6 Build, tests et qualite

Le `pom.xml` inclut :

- `maven-surefire-plugin` pour tests unitaires;
- `maven-failsafe-plugin` pour tests integration;
- `jacoco-maven-plugin` pour couverture;
- profil `analysis` avec SpotBugs et Checkstyle.

6.7 Conteneurisation et CI/CD

Docker

Le `Dockerfile` utilise un build multi-stage :

1. stage Maven pour compiler le jar ;
2. stage runtime Temurin pour executer `app.jar`.

Docker Compose

`docker-compose.yml` definit le service `app`, mappe le port `8091:8091` et monte le volume `uploads`.

GitLab CI/CD

Le pipeline `.gitlab-ci.yml` execute :

- stage `test` ;
- stage `build` ;
- stage `docker-build` avec push registry.

6.8 Point d'attention DB

Le document `README-ONEDRIVE-DB.md` rappelle qu'un lien OneDrive ne remplace pas un serveur DB. Une base relationnelle reelle (MySQL/PostgreSQL) reste obligatoire pour l'execution applicative.

Chapitre 7

Etude du frontend Flutter

7.1 Vue generale

Le frontend est un projet Flutter (SDK Dart ^{3.7.2}) *organisee* en couches :

`components`

`controllers`

`services`

`models`

`screens`

`theme, utils, l10n`

Cette separation facilite la maintenance et l'evolutivite.

7.2 Deux points d'entree applicatifs

Application mobile

Le fichier `lib/main.dart` démarre l'application principale utilisateur, configure theme, locale, routes et ecran home selon l'etat d'authentification.

Plateforme admin web

Le fichier `lib/main_admin_web.dart` démarre une variante web admin avec :

- verification du role admin ;
- ecran dedie `AdminWebPlatformScreen` ;
- message explicite pour les comptes non-admin.

7.3 Navigation, roles et securisation UI

Le frontend utilise des routes nommees et un composant `RoleGuard` pour limiter l'accès aux ecrans sensibles (data management, outils trainer, admin web).

7.4 Theming et coherence visuelle

Le theme global est centralise via `AppTheme.light()` et `AppTheme.dark()`, puis applique dans les deux entrees. Cette approche garantit une coherence visuelle transversale mobile/web.

7.5 Internationalisation

La classe `AppStrings` declare trois locales supportees :

- anglais (en)
- francais (fr)
- arabe (ar)

Le mecanisme de localisation est branche via `localizationsDelegates` dans `MaterialApp`.

7.6 Configuration reseau frontend

Le fichier `Constants.dart` centralise :

- base REST backend : `http://<host>:8091/football-academy/api` ;
- base WebSocket : `ws://<host>:8091/football-academy/ws` ;
- API chatbot locale : `http://localhost:8000/api/chat` (ou 10.0.2.2 sur emulateur Android).

7.7 Ameliorations majeures du module chat

D'apres `CHAT_SYSTEM_IMPROVEMENTS.md`, les evolutions clefs sont :

- passage a un `ChatController singleton` ;
- bootstrap unique puis reutilisation d'etat ;
- pre-subscription de toutes les conversations ;
- mise a jour temps reel conversation/thread sans rechargement ;
- amelioration UX (skeleton loaders, badges, formatting temporel).

7.8 Ameliorations de la page contacts

Le document `CONTACT_PAGE_IMPROVEMENTS.md` introduit :

- chargement role-based des contacts par default ;
- recherche scopee pour trainer ;
- organisation UI par groupes et messages directs ;
- chargement parallele conversations + contacts pour latence reduite.

7.9 Notes de compatibilite

`README_PLATFORM_LIMITS.txt` precise des limites natives pour le scroll vocal system-wide. Les besoins de service en arriere-plan et accessibilite Android depassent la seule couche Flutter `/lib`.

Chapitre 8

Etude du chatbot Django + ML

8.1 Objectif du chatbot

Le projet chatbot fournit un service de question/reponse local pour assister les utilisateurs sur des sujets academie (inscription, frais, planning, contact, etc.).

8.2 Stack technique

Le fichier `requirements.txt` indique une base Django 5.2.1 completee par des outils data science :

- `pandas`
- `numpy`
- `scikit-learn`
- `rapidfuzz`

8.3 Exposition API

Le routage principal est defini par :

- `/` pour l'interface web ;
- `/chat` pour chat web ;
- `/api/chat` pour integration applicative externe.

Le contrat `/api/chat` retourne notamment :

- `response`
- `score`
- `category`
- `source`
- `matched_question`

8.4 Moteur de reponse

Le service `ml_index.py` implemente une strategie en trois niveaux :

1. exact match sur question normalisee ;
2. fuzzy match (RapidFuzz) si exact non trouve ;
3. recherche TF-IDF + cosine similarity via scikit-learn.

Le service `bot.py` combine aussi des intents predefinis avant de passer sur la recherche ML.

8.5 Configuration et tuning

Le fichier `settings.py` expose :

- `QA_CSV` : chemin du dataset question/reponse ;
- `MIN_SIM` : seuil minimal similarity ;
- `FUZZY_MIN` : seuil fuzzy matching ;
- `CHATBOT_API_KEY` : protection optionnelle de `/api/chat`.

8.6 Securite API

Le module `api_auth.py` permet de forcer une cle API via header `X-API-Key`. Cela facilite l'isolation de l'endpoint en environnement partage.

8.7 Integration avec Flutter

Le frontend Flutter utilise `ChatbotService.dart` :

- envoi POST JSON vers `CHATBOT_API_CHAT_URL` ;
- gestion du timeout via `API_TIMEOUT` ;
- support optionnel de `CHATBOT_API_KEY`.

Cette integration garde le chatbot decouple du backend principal, ce qui simplifie son evolution independante.

Chapitre 9

Service IA et Scouting

9.1 Vue d'ensemble

Le service de scouting IA est un microservice FastAPI dedie aux fonctionnalites d'intelligence artificielle et de scouting sportif. Il complete l'ecosysteme en fournissant :

- **ML01** : Score de potentiel sportif avec explications des facteurs
- **ML02** : Analyse de l'evolution sportive (progression, stabilite, regression)
- **ML03** : Prediction du churn (risque d'abandon, raisons probables, actions recommandees)
- **SC01** : Recherche avancee, comparaison des joueurs, generation de shortlists

9.2 Stack technique

Composant	Technologie
Framework API	FastAPI
Langage	Python 3.9+
Base de donnees	SQLAlchemy + SQLite/PostgreSQL
ML/Data Science	scikit-learn, pandas, numpy
API Documentation	Swagger UI (auto-generee)
Tests	pytest + pytest-asyncio

TABLE 9.1 – Synthese stack service IA

9.3 Architecture du service

Structure modulaire

```

app/
|-- main.py          # Point d'entree FastAPI
|-- api/
|   |-- routes/
|   |   |-- health.py    # Health check
|   |   |-- data.py      # Data ingestion endpoints
|   |   |-- ml.py        # ML scoring endpoints
|   |   |-- scouter.py   # Scouting features
|-- services/
|   |-- ml_service.py    # Logique ML
|   |-- feature_engineering.py
|   |-- sync_service.py  # Sync avec football-academy
|-- db/
|   |-- models.py        # Modeles SQLAlchemy
|   |-- session.py       # Gestion session DB
|-- tests/
|   |-- test_scoring.py  # Tests unitaires ML

```

9.4 Fonctionnalites ML

ML01 - Score de potentiel sportif

L'endpoint GET `/api/v1/ml/potential/{player_external_id}` calcule un score de potentiel base sur :

- Statistiques de performance (buts, passes decisives, notes moyennes)
- Assidueite aux entrainements
- Historique des blessures
- Age et position

Reponse API :

```

{
  "player_external_id": 101,
  "potential_score": 78.5,
  "factors": {
    "performance": 82.0,
    "attendance": 75.0,
    "health": 80.0,
    "age_factor": 76.0
  },

```

```
"explanation": "Fort potentiel offensif avec bonne assiduite"
}
```

ML02 - Analyse de l'évolution

L'endpoint `GET /api/v1/ml/evolution/{player_external_id}` analyse la tendance sur une fenetre temporelle configurable :

- **Progression** : Amelioration significative des performances
- **Stabilite** : Performances constantes
- **Regression** : Baisse des performances

ML03 - Prediction du churn

L'endpoint `GET /api/v1/ml/churn/{player_external_id}` estime le risque d'abandon :

```
{
  "player_external_id": 101,
  "churn_probability": 0.35,
  "risk_level": "medium",
  "likely_reasons": [
    "payment_delays",
    "decreasing_attendance"
  ],
  "recommended_actions": [
    "contact_parent",
    "offer_payment_plan",
    "schedule_meeting"
  ]
}
```

9.5 Fonctionnalites Scouting

SC01 - Recherche avancee

L'endpoint `GET /api/v1/scouter/players/search` permet une recherche multi-criteres :

- Filtrage par position, age, division
- Seuils sur potentiel minimum et churn maximum
- Tri par differentes metriques

Exemple de requete :

```
GET /api/v1/scouter/players/search?
    position=MID&
    min_potential=65&
    max_churn=0.4&
    limit=20
```

Generation de shortlists

L'endpoint `POST /api/v1/scouter/shortlists/generate` cree des listes de joueurs selon differentes strategies :

- **balanced** : Equilibre entre potentiel et faible risque
- **potential** : Priorite au potentiel pur
- **safe** : Priorite aux joueurs a faible risque

9.6 Integration avec football-academy

Synchronisation des donnees

L'endpoint `POST /api/v1/data/sync/football-academy` synchronise automatiquement :

- Donnees joueurs depuis `/football-academy/api/players`
- Donnees paiements depuis `/football-academy/api/payments`

Configuration de l'integration

```
# .env
FOOTBALL_BACKEND_BASE_URL=http://localhost:8091
SYNC_INTERVAL_HOURS=24
```

9.7 Tests et qualite

Tests unitaires

Les tests couvrent :

- Calcul des scores de potentiel
- Analyse d'evolution

- Prediction de churn
- Recherche et filtrage

Validation des modeles

Les modeles ML sont valides via :

- Cross-validation
- Mesures de performance (Accuracy, F1, MAE, RMSE)
- Analyse des features importantes

9.8 Deploiement

Dockerisation

Le service est conteneurise avec Docker :

```
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install -r requirements.txt
COPY . .
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8010"]
```

Integration Docker Compose

Le service s'integre dans l'écosysteme via docker-compose.yml :

```
services:
  scouting-ai:
    build: ./scouting-ai-service
    ports:
      - "8010:8010"
    environment:
      - FOOTBALL_BACKEND_BASE_URL=http://football-academy:8091
    depends_on:
      - football-academy
```


Chapitre 10

Fonctionnalites Multi-sport

10.1 Concept et architecture

La plateforme *Sport Academy Pro* est concue pour etre multi-sport, permettant a une seule application de gerer differents sports (football, basketball, handball, speedball, etc.) sans duplication de code.

10.2 Principes de conception

Noyau commun

Un noyau commun gere les fonctionnalites transversales :

- Authentification et gestion des utilisateurs
- Systeme de roles et permissions
- Calendrier et planning
- Messagerie et notifications
- Paiements et cotisations
- Back-office administratif

Elements configurables

Chaque sport dispose de configurations specifiques :

- Positions et roles sur le terrain
- Statistiques et indicateurs de performance
- Regles et configurations specifiques
- Terminologie et labels

10.3 Implementation technique

Modele de donnees multi-sport

```
// Entite Sport
@Entity
public class Sport {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;           // "Football", "Basketball", etc.
    private String code;           // "FB", "BK", etc.
    private boolean isActive;

    @OneToMany(mappedBy = "sport")
    private List<Position> positions;

    @OneToMany(mappedBy = "sport")
    private List<StatDefinition> statDefinitions;
}

// Entite Position (configurable par sport)
@Entity
public class Position {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;           // "Gardien", "Attaquant", etc.
    private String code;           // "GK", "FW", etc.
    private String abbreviation;

    @ManyToOne
    private Sport sport;
}

// Definition des statistiques par sport
@Entity
public class StatDefinition {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
```

```

private Long id;

private String name;           // "Buts", "Passes decisives"
private String code;           // "goals", "assists"
private String dataType;       // "integer", "decimal", "time"

@ManyToOne
private Sport sport;
}

```

Configuration dynamique

Les configurations sont chargees dynamiquement selon le sport actif :

```

@Service
public class SportConfigService {

    public List<Position> getPositions(Long sportId) {
        return positionRepository.findBySportId(sportId);
    }

    public List<StatDefinition> getStatDefinitions(Long sportId) {
        return statDefinitionRepository.findBySportId(sportId);
    }

    public SportConfig getSportConfig(Long sportId) {
        Sport sport = sportRepository.findById(sportId)
            .orElseThrow(() -> new SportNotFoundException(sportId));

        return SportConfig.builder()
            .sport(sport)
            .positions(getPositions(sportId))
            .statDefinitions(getStatDefinitions(sportId))
            .build();
    }
}

```

10.4 Fonctionnalités multi-sport

MS01 - Activation ou ajout d'un sport

L'administrateur peut activer ou désactiver des sports via le back-office :

- Liste des sports disponibles
- Activation/désactivation en un clic
- Configuration initiale des positions et statistiques

MS02 - Configuration par sport

Chaque sport peut être configuré indépendamment :

- Positions spécifiques (ex : gardien, défenseur pour le football)
- Statistiques pertinentes (buts pour football, points pour basketball)
- Règles de calcul et indicateurs
- Terminologie et labels

MS03 - UX unifiée

L'interface utilisateur reste la même quel que soit le sport :

- Même navigation et structure
- Seules les données et labels changent
- Adaptation automatique des formulaires
- Filtres et recherches contextuels

10.5 Exemples d'implémentation

Football

Element	Configuration
Positions	Gardien, Défenseur, Milieu, Attaquant
Statistiques	Buts, Passes décisives, Tacles, Interceptions
Indicateurs	Note moyenne, Minutes jouées, Cartons

TABLE 10.1 – Configuration Football

Basketball

Element	Configuration
Positions	Meneur, Arriere, Ailier, Pivot
Statistiques	Points, Rebonds, Passes decisives, Interceptions
Indicateurs	Pourcentage de tir, Plus/minus, Temps de jeu

TABLE 10.2 – Configuration Basketball

10.6 Avantages de l’approche multi-sport

- **Maintenance simpliffee** : Un seul code base a maintenir
- **Extensibilite** : Ajout facile de nouveaux sports
- **Consistance** : UX uniforme pour tous les sports
- **Economie d’echelle** : Developpement et maintenance optimises

10.7 Integration avec les services IA

Les services IA s’adaptent automatiquement au sport :

- Calcul du potentiel selon les statistiques du sport
- Analyse d’evolution contextuelle
- Prediction de churn adaptee
- Recherche scouting filtree par sport

Chapitre 11

Integration et deploiement

11.1 Integration inter-projets

Frontend <-> Backend

Le frontend Flutter consomme l'API backend sur :

- REST : `http://<host>:8091/football-academy/api`
- WebSocket STOMP : `ws://<host>:8091/football-academy/ws`

Frontend <-> Chatbot

Le module chatbot Flutter appelle le service Django sur `http://localhost:8000/api/chat` (ou 10.0.2.2 sur emulateur Android).

11.2 Variables de configuration

Variable	Usage
PORT	Port backend Spring Boot (defaut 8091).
DB_URL, DB_USER, DB_PASS	Connexion base de donnees backend.
CHATBOT_API_KEY	Protection optionnelle endpoint chatbot.
QA_CSV	Dataset Q/R du chatbot.
MIN_SIM, FUZZY_MIN	Parametrage du moteur de similarite chatbot.

TABLE 11.1 – Variables principales de deploiement

11.3 Strategie de deploiement recommandee

Backend

Deploiement conteneurise via Docker, pilote par pipeline CI/CD (test/build/docker push).

Frontend mobile

Build et distribution independants pour mobile (Android/iOS).

Frontend admin web

Build web separe et hebergement sur une plateforme statique ou serveur web. Cette separation evite de lier les contraintes web admin au package mobile.

Chatbot

Deploiement en service Python separable (VM/container) selon la charge. En local, execution possible sur la meme machine que le frontend.

11.4 Procedure de lancement local

Backend Spring Boot

```
cd football-academy
mvn clean package
mvn spring-boot:run
```

Frontend Flutter

```
cd moez_project
flutter pub get
flutter run
```

Chatbot Django

```
cd chatbot/chatbot
python -m venv env_py10
# activation env selon systeme
```

```
pip install -r requirements.txt  
python manage.py migrate  
python manage.py runserver 127.0.0.1:8000
```

11.5 Points de vigilance en production

- externaliser les secrets (JWT, DB, API keys) hors du code source ;
- vérifier les CORS et restrictions de domaines autorisés ;
- monitorer les performances chat temps réel et chatbot ;
- mettre en place des sauvegardes DB et retention logs.

Chapitre 12

Tests et assurance qualite

12.1 Qualite backend

Le backend integre nativement plusieurs leviers qualite :

- tests unitaires via Surefire ;
- tests integration via Failsafe ;
- couverture via JaCoCo ;
- analyse statique via SpotBugs et Checkstyle (profil **analysis**).

Le pipeline GitLab automatise ces etapes avant la creation/push d'image Docker.

12.2 Qualite frontend

Le frontend Flutter peut etre verifie avec :

- `flutter analyze` pour l'analyse statique ;
- `flutter test` pour tests unitaires/widgets ;
- tests manuels sur les parcours critiques (auth, navigation role-based, chat, notifications).

12.3 Qualite chatbot

Pour le chatbot, trois niveaux de verification sont recommandes :

- tests fonctionnels de l'API `/api/chat` ;
- evaluation precision sur un jeu de questions de reference ;
- suivi des scores similarity pour ajuster `MIN_SIM` et `FUZZY_MIN`.

12.4 Scenario de validation bout en bout

1. Connexion utilisateur Flutter et recuperation JWT.
2. Chargement des donnees metier depuis le backend.
3. Ouverture du module chat et verification des updates temps reel.
4. Appel chatbot et verification du format de reponse.
5. Verification des restrictions de role sur ecrans admin.

12.5 Indicateurs de suivi proposes

Indicateur	Objectif
Taux de succes API backend	Mesurer la stabilite des endpoints.
Temps de reponse chat temps reel	Valider la fluidite conversationnelle.
Precision chatbot (Top-1)	Evaluer la pertinence des reponses.
Taux erreurs UI critiques	Reduire les regressions fonctionnelles.

TABLE 12.1 – KPIs qualite recommandes

Chapitre 13

Limites actuelles et pistes d'amélioration

13.1 Limites identifiées

Limites plateforme mobile

Le document `README_PLATFORM_LIMITS.txt` signale que certaines fonctions system-wide (ex : scroll vocal hors app) nécessitent du natif Android (ForegroundService + AccessibilityService), non couvert uniquement par la couche Flutter `/lib`.

Dependance chatbot locale

En mode local, le chatbot est configure sur `localhost:8000`. Sans service actif, l'écran chatbot doit afficher une dégradation contrôlée (message d'erreur clair et non bloquant).

Couverture endpoints contacts role-based

Le document `CONTACT_PAGE_IMPROVEMENTS.md` décrit des endpoints role-spécifiques cibles. Leur alignement complet backend/frontend doit être vérifié en continu.

13.2 Améliorations prioritaires

1. Mettre en place une matrice de tests E2E par rôle (admin, trainer, parent, player).
2. Industrialiser le déploiement du chatbot (container dédié, health checks, monitoring).
3. Renforcer observabilité backend (metrics, tracing, dashboards).
4. Ajouter une stratégie de gestion des secrets centralisée (vault ou équivalent).
5. Standardiser les conventions de code/lint pour réduire la dette technique.

13.3 Evolutions fonctionnelles chat/chatbot

D'après les notes d'amélioration existantes, évolutions intéressantes :

- indicateurs de frappe et accusés de lecture enrichis ;
- édition/suppression de messages ;
- recherche full-text conversationnelle ;
- support media (image, fichier, video) ;
- enrichissement continu du dataset chatbot.

13.4 Perspective architecture

A moyen terme, une évolution vers une architecture plus modulaire peut être envisagée :

- API gateway ;
- séparation claire service notifications/chatbot ;
- cache distribuée pour accélérer les lectures fréquentes.

Chapitre 14

Conclusion generale

Ce travail a permis de structurer un rapport technique complet autour de trois composantes complementaires :

- un backend Spring Boot robuste, securise et conteneurisable ;
- un frontend Flutter moderne avec separation mobile/web admin ;
- un chatbot Django/ML integre par API et configurable par seuils de pertinence.

L'analyse documentaire montre une base solide pour la poursuite du projet en production, notamment grace a :

- la presence d'un pipeline CI/CD backend ;
- la centralisation des configurations reseau et des themes cote Flutter ;
- la modularite du moteur chatbot et son integration explicite.

Les prochaines etapes recommandees concernent surtout l'industrialisation (monitoring, tests E2E, securite secrets) et l'approfondissement fonctionnel (chat enrichi, precision chatbot, optimisation UX multi-role).

En resume, l'ecosysteme actuel constitue une base credible et evolutive pour un produit academique/professionnel de gestion d'academie de football.

Chapitre 15

Annexes

15.1 Annexe A - Commandes utiles

Backend

```
cd football-academy
mvn clean test
mvn clean package -DskipTests
mvn spring-boot:run
```

Frontend

```
cd moez_project
flutter pub get
flutter analyze
flutter test
flutter run
```

Chatbot

```
cd chatbot/chatbot
python -m venv env_py10
pip install -r requirements.txt
python manage.py migrate
python manage.py runserver 127.0.0.1:8000
```

15.2 Annexe B - Exemple de payload chatbot

```
{  
  "message": "How can I contact you?",  
  "sender_id": "optional"  
}
```

```
{  
  "response": "...",  
  "score": 0.42,  
  "category": "Support/Contact",  
  "source": "Academy Info",  
  "matched_question": "How can I contact you?"  
}
```

15.3 Annexe C - Emplacements a completer dans le rapport

- couverture : nom etablissement, encadrants, auteur ;
- captures d’ecran UI dans **figures/** ;
- resultats de tests reels (latence, precision chatbot, couverture code) ;
- diagrammes UML/BPMN si requis par le cahier de stage.